

XI. Lab 11: Microcontroller applications

A. Introduction

Logic circuits can be built from low-level ICs such as logic gates, multiplexers, decoders, flip-flops, and registers like the ones we worked with in Labs 9 and 10. However in practice this approach is used only for the simplest circuits. For more complex circuits, designers turn to one of the following larger-scale integrated circuit solutions.

1. Application-Specific Integrated Circuit (ASIC)

An ASIC is a large-scale integrated circuit that is custom designed to include all functions needed for some task. ASICs can have analog functions, digital functions, or both. ASICs are often used to receive, process, and transmit data in consumer electronics, in telecommunications equipment, or for automotive or medical applications. The cost to produce a small number of ASICs for a given design is very high, and for this reason they're not well-suited to niche or low-volume applications. But, once the design and production process for an ASIC is complete, they can be very cost-effective in markets where large numbers of the same ASIC are needed.

2. Programmable Logic

The two main types of programmable logic are Complex Programmable Logic Devices (CPLDs) and Field Programmable Gate Arrays (FPGAs). These devices contain arrays of logic elements such as gates and registers containing flip-flops. They also include programmable connections between the logic elements. This allows a wide variety of digital circuits to be constructed on a given CPLD or FPGA by programming of the connections.

One difference between CPLDs and FPGAs is that FPGAs have larger numbers of smaller logic elements and can be configured in more complex arrangements than CPLDs. Also the programming of a CPLD is stored in non-volatile memory, while an FPGA must go through a boot process when it is switched on. CPLDs are also less expensive than FPGAs.

Small FPGAs have a few thousand logic gates, while the largest have billions of gates. Thus they are capable of extremely complex operations. CPLDs and FPGAs are well suited to the production of complex logic circuits in which the programming can be fixed and in which logic speed is a priority. Their advantage over ASICs is the avoidance of the initial high cost of design and production of a customized IC.

3. Microcontrollers

A Microcontroller is a device containing a CPU, memory, clock, and various input and output devices. It is typically used as an embedded controller for another piece of equipment. Computers with associated peripherals can also carry out such control functions, but microcontrollers are much smaller, more integrated, and less expensive. Also, microcontrollers typically have a much more limited instruction set than a computer and are programmed through an Integrated Development Environment (IDE) on another connected device. Also, their clock speed is typically 100 MHz or less.

Microcontrollers and FPGAs with associated peripherals are both well-suited to control tasks. FPGAs are generally better for demanding tasks requiring a high level of customization, complex programming, and high speed. Microcontrollers are less customizable and slower than FPGAs, but are also much less expensive and can be easier to work with.

4. Arduino UNO R4 Minima Microcontroller

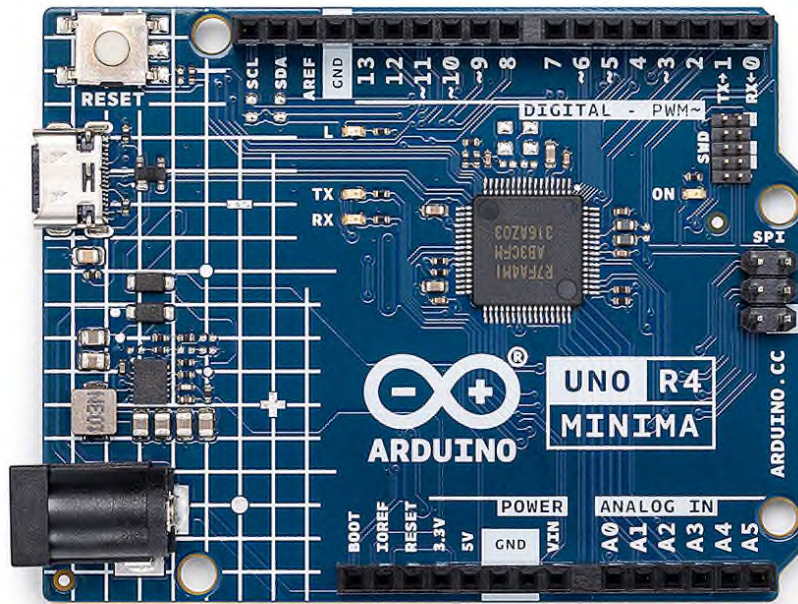


Figure XI-1. Arduino UNO R4 Minima microcontroller.

In this lab, we'll explore applications of one of these devices, the microcontroller. We'll carry out some basic tasks with the [Arduino UNO R4 Microcontroller](#) shown in Figure XI-1. This microcontroller has the following features;

- Renesas RA4M1 (Arm Cortex-M4) 32 bit microprocessor with 48 MHz clock
- 256kB Flash Memory, 32kB RAM
- USB-C programming port
- 14 digital input/output pins
- 6 digital outputs that support Pulse Width Modulation (PWM)
- On-board UART, I2C, SPI, and CAN communication peripherals
- 6 analog input pins
- 1 Digital to Analog Converter (DAC) for analog output
- SWD connector for debugging, HID support

B. Procedure

The Arduino UNO R4 Minima is very well documented. The documents are too long to include here, but you can find them through this Arduino web site: <https://docs.arduino.cc/hardware/uno->

[r4-minima/](#). I recommend that you pull up this web site on your laptop so that you can refer to it while working.

1. Getting started with the Arduino IDE

Navigate to this web site: <https://docs.arduino.cc/tutorials/uno-r4-minima/minima-getting-started/> Click on the link “Arduino software page”. Select your computer type, download the IDE, and install it. Install the additional programs as requested. Then run the IDE. Next you’ll need to install software to run the UNO R4 board. To do this, click on the “board manager” icon in the IDE (second one from the top in the left hand column). Scroll down through the boards until you come to “Arduino UNO R4 boards” and click “install.” When the installation is complete, connect your Arduino board to the computer with a USB cable. In the “select board” window, select your Arduino UNO R4.

To work with your board, you enter a “sketch” into the IDE window and then upload it to your board. A “sketch” is an Arduino term for a program. The IDE has some built-in sketches. To see how this works, go to File → Examples → Basics → Blink. The IDE will open a new window with the Blink sketch in it.

Study the program so you see how it works. In the code the greyed-out text gives comments that don’t affect execution. “void” gives the return type of the function that follows. “void” means that the function doesn’t return anything. “setup()” is a function that runs only once at the beginning of a sketch to initialize settings and variables. This function is defined in the lines that follow. In this case the function is “pinMode”. In the Arduino, digital pins can be either inputs or outputs. In this case `pinMode(LED_BUILTIN, OUTPUT);` tells the Arduino that the built-in LED should be an output. The next statement “void loop()” tells the Arduino to execute the following instructions in a repeating loop. Again “void” indicates that this function doesn’t return anything. Inside the loop the instructions set the LED output high and then low with times delays. The unit for the delay time is ms, so `delay(1000)` delays for 1 s.

The next step in using your code is to verify it. To do this, click the verify icon (check mark in the upper left). The IDE will compile your code and notify you of any errors. To see how this part work, first try deleting one of the semicolons at the end of a line followed by verify, and then replace the semicolon and run verify the code again. The semicolon is required to terminate and instruction. In the first case verify will notify you of the error, while in the second case the code will compile without any errors found.

Once your code compiles you can upload it to the Arduino by clicking on the “upload” icon (right arrow next to the check). Do this with your successfully verified “blink” code. You should now see the program running on your Arduino, *i.e.* you should see the LED on the board blinking on and off.

Next, try editing the code so that instead of being on for one second and off for one second. The LED is on and off for some noticeably different period of time. Verify and upload your new code. You should now see the LED blinking with your new time periods.

2. Connecting a display

Your Arduino can drive displays with a variety of communication protocols. Here, we'll drive a SunFounder CN0295D16x2 alphanumeric display with the I2C protocol. Directions for how to do this with your Arduino are attached to the end of this section. Following these directions, connect your display to the Arduino and get it to output text on the display. You can make the connections with male-to-female jumper wires. Also, note that if any instruction that is on one line must remain on one line in the IDE. If it is split into two lines it will generate an error. Finally, if you wish you can adjust the contrast of your display using the blue screw on the back.

Change the program to output a few of your favorite words and verify that these new words are output on the display.

3. Measuring an analog voltage

In this part of the lab we will use an Arduino analog input to measure an analog signal. As is usual with electronics, the analog voltage range cannot go beyond the range of the supplies. In this case it means that the analog input voltage must remain between 0 Volts and 5 Volts. Inputting a voltage outside that range runs the risk of damaging the Arduino, so please keep your input voltage between 0 Volts and 5 Volts.

Set up a function generator to display on an oscilloscope and to also apply the output voltage to breadboard wires. Adjust the function generator to output a sine wave with 20 Hz frequency, 1 Volt amplitude, and 2 Volts offset. In other words, the sine wave output should vary between 1 Volt and 3 Volts. This will satisfy the above requirement on input voltage range.

Go to File → Examples → Basics → ReadAnalogVoltage. Then, click on Tools and select "Serial Plotter." Serial plotter is a tool that you can use within the Arduino programs to display a voltage vs. time. ReadAnalogVoltage is a program that will read in a voltage from the Arduino analog input 0 and display it on the serial plotter. Read through the program and try to understand how it works.

Hook up the signal from your function generator to the Arduino. The signal should be connected to Analog In A0 and the ground should be connected to GND. You should see the sine wave output of the function generator on the serial plotter. Verify that the signal is the same as what you see on the oscilloscope.

Next, increase the frequency of the generator and watch what happens on the serial plotter window. You should find at some point that the sampling rate of the analog signal is not fast enough to keep up with the changing sine wave. (You may see an effect called "aliasing", which happens due to periodic sampling of a periodic wave. This can result in apparent high frequency "sine waves". However if you tune the frequency in this condition you won't see the sine waves change continuously in frequency on the display.) At about what frequency does the display fail to keep up with the input sine wave?

Next see if you can get the Arduino to record a faster sine wave. To do this, you'll need to change the serial communication rate in both the sketch window and in the serial plotter. The two rates need to be the same. Try a faster rate such as 250000 bits per second. Verify that you can

now see a faster sine wave on the display. What's the fastest sine wave that you can faithfully record now?

Note: it's possible that trying to change the serial rate will cause the Arduino to fail to load the program. If this happens, pull the USB plug out, then plug it back in, then quickly push the reset button twice. The yellow "L" LED should start flashing. If you need to edit the serial rate, make sure to do that *before* you upload the program. At this point you should be able to upload `ReadAnalogVoltage` and run it at the higher serial rate.

Aside: we've used this method mainly for its simple programming. Using other methods it's possible to make much faster measurements with the Arduino.

4. Digital PID control loop

Next, we'll explore the use of the Arduino to implement a digital PID control loop. This exercise is taken from this source:

YouTubeVideo: <https://www.youtube.com/watch?v=RZW1PsfgVEI&t=267s>

Github description: <https://gist.github.com/careyi3/02a57dfd3a62a96d46171489c83488bd>

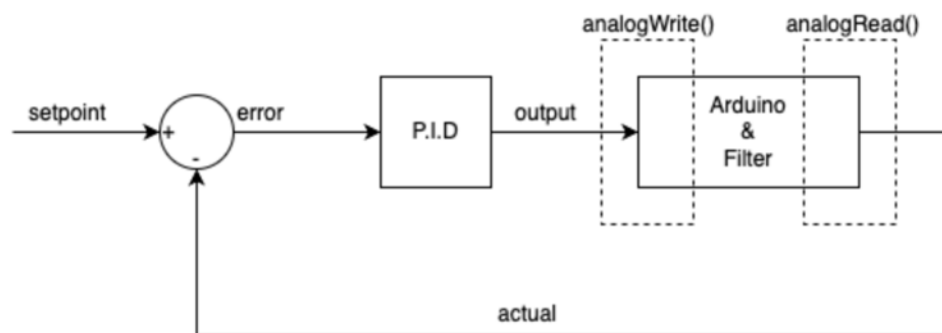


Figure XI-2. PID control loop

A PID control loop is a system that is designed to stabilize a measured parameter x of a system to a setpoint value x_{set} . To do this, x is measured electronically. A circuit produces a voltage ε proportional to the difference between x and x_{set} . This is called the *error signal*. The error signal is then fed into a *loop filter*. The loop filter applies a transformation to the error signal and its output is fed to a transducer that can change the value of x . The most common loop filter is the *PID* filter. In such a filter, the output is the sum of a voltage proportional to ε (the "P") a voltage proportional to the time integral of ε (the "I"), and a term proportional to the time derivative of ε (the "D"). The output of the loop filter together with the transducer produces a change in the value of x that moves it closer to x_{set} . Depending on the amounts of P, I, and D in the loop filter, the PID control loop can be highly effective in stabilizing x to x_{set} , moderately effective in stabilizing x to x_{set} , or completely ineffective. In some situations the control loop can be completely unstable and send the value of x into large oscillations in time.

The loop filter can be produced either with analog or digital circuits. In this exercise we'll work through a simple example of using a digital PID control of a voltage.

You may wish to refer to the above YouTube video or to the Github description for further details.

To get started, construct the following circuit using a breadboard next to your Arduino:

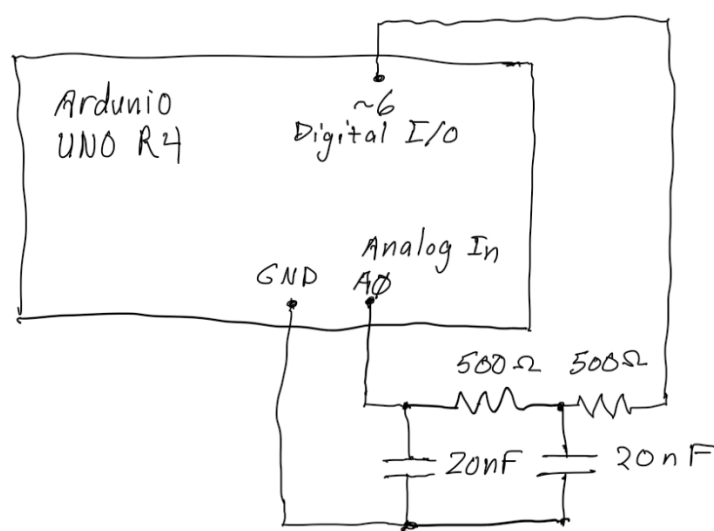


Figure XI-3. Circuit for PID control loop demonstration

Next, upload the following program to your Arduino.

```
const int INPUT_PIN = A0;
const int OUTPUT_PIN = D6;

double dt, last_time;
double integral, previous, output = 0;
double kp, ki, kd;
double setpoint = 75.00;

void setup()
{
  kp = 0.8;
  ki = 0.20;
  kd = 0.001;
  last_time = 0;
  Serial.begin(9600);
  analogWrite(OUTPUT_PIN, 0);
  for(int i = 0; i < 50; i++)
  {
    Serial.print(setpoint);
    Serial.print(",");
    Serial.println(0);
    delay(100);
  }
}
```

```

    delay(100);
}

void loop()
{
    double now = millis();
    dt = (now - last_time)/1000.00;
    last_time = now;

    double actual = map(analogRead(INPUT_PIN), 0, 1024, 0, 255);
    double error = setpoint - actual;
    output = pid(error);

    analogWrite(OUTPUT_PIN, output);

    // Setpoint VS Actual
    Serial.print(setpoint);
    Serial.print(",");
    Serial.println(actual);

    // Error
    //Serial.println(error);

    delay(300);
}

double pid(double error)
{
    double proportional = error;
    integral += error * dt;
    double derivative = (error - previous) / dt;
    previous = error;
    double output = (kp * proportional) + (ki * integral) + (kd * derivative);
    return output;
}

```

This circuit and program attempts to stabilize the voltage output on digital I/O pin 6 to the setpoint value of 75. It uses pulse-width modulation to produce an analog voltage on the digital output. This works by varying the width of a series of output pulses. The capacitors and resistors form a low-pass filter. This removes the discrete pulses from the output voltage and produces a smoothed analog output. This output voltage is proportional to the widths of the digital pulses. This method of producing an analog voltage is referred to as Pulse Width Modulation (PWM).

Finally, open a serial plotter window with the baud rate set to 9600, and view the resulting voltages. The two lines show the output analog voltage and the set-point voltage vs. time. To see the response of the circuit to a large step in voltage, press the reset button and wait. You should see the voltage drop to zero for a period of time, after which the feedback loop kicks in. What happens as the feedback tries to stabilize the voltage to the set-point?

Using the oscilloscope, observe the direct digital output at I/O pin 6, and also view the filtered output. In your report, comment on how effective the filter is at removing the PWM from the output.

The P, I, and D feedback in the loop is set by the parameters k_p , k_i , and k_d . Try changing these parameters and see what happens. (You'll need to re-upload your program each time you change the parameters. Also, you should press reset to observe the initial response of the loop for each new set of parameters.

Record what happens for some representative set of parameters. What parameters cause the voltage to converge most quickly to the set-point? Under what conditions is the voltage vs time more stable? Less stable? Completely unstable?

C. SunFounder Display Directions

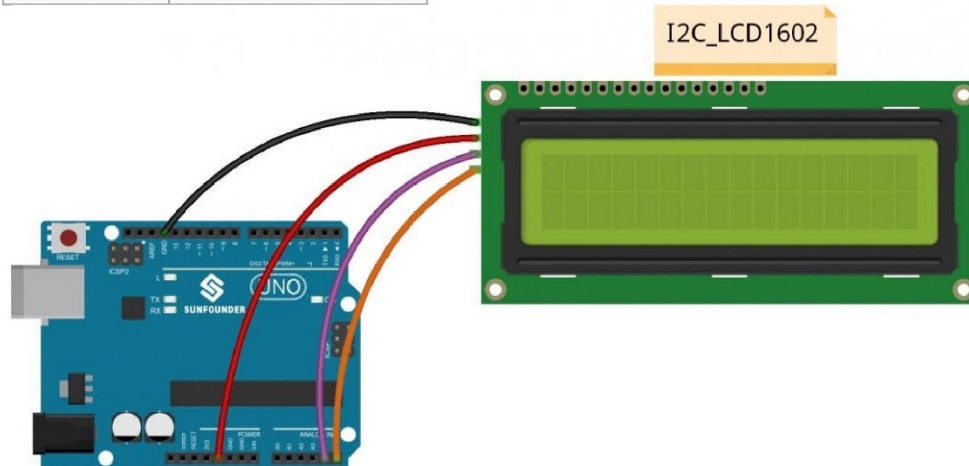
These are only abbreviated directions for connecting the display to the Arduino. For full directions see

<https://mm.digikey.com/Volume0/opasdata/d220001/medias/docus/5773/CN0295D%20DATASHEET.pdf>

How to Use in Arduino?

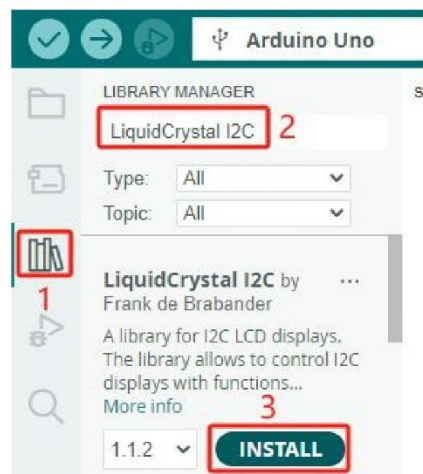
Build the Circuit

I2C LCD1602	Arduino Board
GND	GND
VCC	5V
SDA	A4 /pin 20 mega2560
SCL	A5 /pin 21 mega2560



Install the Library

Open the **Library Manager**, enter **LiquidCrystal I2C**, and click **INSTALL** when it appears.



Copy and Upload the Code

1. Copy and paste the following code to the Arduino IDE.

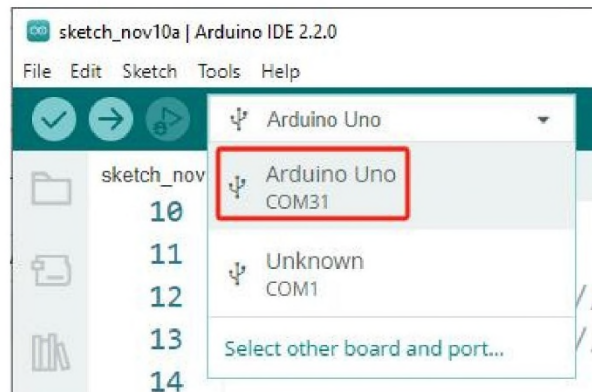
```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
/*****

LiquidCrystal_I2C lcd(0x27, 16, 2); // set the LCD address to 0x27 for a 16 chars and 2 line display
*****/

void setup()
{
  lcd.init(); //initialize the lcd
  lcd.backlight(); //open the backlight
}
/*****

void loop()
{
  lcd.setCursor(3, 0); // set the cursor to column 3, line 0
  lcd.print("SunFounder"); // Print a message to the LCD
  lcd.setCursor(2, 1); // set the cursor to column 2, line 1
  lcd.print("Hello, World!"); // Print a message to the LCD.
}
*****/
```

2. Choose the board and port you use.



3. Click the **Upload** button.



4. You should now see your I2C LCD1602 display "SunFounder" and "hello, world".



